

Digital Logic Design Report

Overdrive

Group Members:

Azyan Ahmed, Arham Hussain,
Bareera Junaid, Syeda Fatima Alam

Instructor: Farhan Khan



Spring 25'

Date: 15th April, 2025

Index

Page 3- Introduction

Page 5- Objectives

Page 5- Gameplay

Page 6- Input

Page 8- Implementation

Page 9- Code snippets

Page 12- FSM Design

Page 13- VGA Output

Page 15- Results

Page 16- Testing and Simulation

Page 16- Challenges Faced

Page 16- Future Improvements

Page 17- Conclusion

Introduction

Our project named as ‘Overdrive’ focuses on designing and implementing a car racing game on the Basys 3 FPGA board using Verilog, combining both digital logic design and hardware interfacing to recreate the feel of a traditional arcade-style racing experience. The game simulates a race track displayed via VGA output on a monitor, where a car can be navigated through various checkpoints within a set time limit. The player's objective is to complete as many laps as possible, or finish the game within a set time to track their performance through a score counter.

A unique aspect of our project is that we are developing a physical model of an arcade machine to house and present the game. This physical model includes a set of hardware controls resembling a real arcade cabinet, complete with steering mechanisms, pedals, and visual feedback. Keyboard controls, being printed using 3D Printer, are used for steering the car during the game, while foot pedals are incorporated for acceleration and braking, mimicking a real driving experience.

To improve the precision and responsiveness of user inputs, we are integrating light sensors and laser sensors into the control mechanisms. These sensors will detect player actions with greater accuracy compared to conventional buttons alone, providing smoother transitions and enhancing gameplay immersion.

Overall, the goal of our project is not just to build a game but to create a fully interactive, physical arcade gaming system that merges the fun of digital gameplay with the tactile feedback of real-world controls bridging hardware and software in a compact and entertaining form.

Objectives

- Use Verilog to design and simulate a digital game.
- Implement a state machine and multiple modules for VGA output, game logic, timers, and control.
- Learn how to interface the FPGA with VGA and 7-segment display.
- Design a functional, interactive racing game with an intuitive user experience.

Gameplay

- The player controls a car using the direction buttons (up, down, left, right).
- The car moves along a rectangular track.
- A checkpoint must be crossed before the finish line is valid for completing a lap.
- The player has set seconds to complete as many laps as possible.
- There are two modes of the game:
 - Lap Mode: the player that finishes the requires amount of laps will win
 - Time Mode: the player that reaches the lap first in required time will win
- The game ends displaying the final score.

Input

Explanation:

Our car racing game implemented on the Basys 3 FPGA utilizes a combination of input mechanisms to ensure responsive and accurate gameplay. The following input devices are integrated into the system.

The keyboard serves as the primary controller for navigating the car on-screen. It is connected to the Basys 3 board via the PS/2 port. This setup allows intuitive directional control, mimicking the steering of a car.

Pedal Input

We also include foot pedals as a secondary input mechanism to simulate acceleration and braking. Right Pedal (Accelerator) controls the car's forward speed. Left Pedal (Brake) slows down or stops the car. The pedal inputs are read via the sensors and connected to the FPGA through appropriate sensor modules or ADC components.

Light Sensors/ Light Sensors

These sensors enhance positional accuracy. They also help detect lap completions and enforce checkpoint tracking.

These combined inputs: keyboard, pedals, and sensors allow for an immersive, realistic racing experience and accurate game logic implementation.

Input Keys on Keyboard

For Player 01:

Button 2	Move down
Button 4	Move left
Button 6	Move right
Button 8	Move up

For Player 02:

Button X	Move down
Button A	Move left
Button D	Move right
Button A	Move up



Implementation Details

- **VGA Controller:** Generates 640x480 display with 25MHz clock using synchronization signals (HSYNC, VSYNC).
- **Car Controller:** Updates the car's position based on button inputs and prevents out-of-bounds movement.
- **Track Renderer:** Draws the track, checkpoint, and finish line using pixel coordinates.
- **Lap Counter:** Keeps track of valid laps completed and outputs to 7-segment display.
- **Timer Module:** set-second countdown implemented using clock dividers and counters.
- **Score Display:** Displays final lap count after time runs out.

Code Snippets:

Clock

```
module clk_divider(  
    input wire clk_100MHz,  
    output reg clk_25MHz = 0  
);  
    reg [1:0] counter = 0;  
  
    always @(posedge clk_100MHz) begin  
        counter <= counter + 1;  
        if (counter == 2'b11)  
            clk_25MHz <= ~clk_25MHz;  
    end  
endmodule
```

Debounce

```
module debouncer (  
    input wire clk,           // 100MHz clock  
    input wire btn,          // Raw button input  
    output reg btn_clean     // Debounced output  
);  
    reg [19:0] count = 0;    // Counter for ~10ms  
    reg btn_reg = 0;         // Previous button state  
  
    always @(posedge clk) begin  
        btn_reg <= btn;  
        if (btn_reg != btn_clean) begin  
            count <= count + 1;  
            if (count == 20'hFFFFFF) begin // ~10ms  
                btn_clean <= btn_reg;  
                count <= 0;  
            end  
        end  
        else  
            count <= 0;  
    end  
endmodule
```

Pixel Generator

```

module pixel_generator(
    input wire clk,           // Clock input from top file
    input wire reset,         // Reset input from top file
    input wire p_tick,        // Pixel tick from vga_sync
    input wire video_on,       // Video on signal from vga_sync
    output reg [11:0] rgb      // 12-bit RGB output: {Green[3:0], Blue[3:0], Red[3:0]}
);

    always @* begin
        if (~video_on)
            rgb = 12'h000; // Black during blanking
        else
            rgb = 12'hFFF; // White when video_on
        end

    // Commented out unnecessary game-specific inputs and logic
    /*
    input wire [9:0] x, y,      // Pixel coordinates (available but not used)
    input wire [9:0] car_x, car_y, // Car position
    input wire [7:0] lap_counter, // Lap counter
    input wire [5:0] timer,      // Timer
    input wire game_over,        // Game over flag

    parameter CAR_SIZE = 5;
    parameter CHECKPOINT_X = 100;
    parameter FINISH_X = 540;

    // Digit bitmap ROM (for displaying numbers)
    reg [7:0] digit_bitmap [0:9][0:7];

    // Initialize digit bitmaps (0-9)
    initial begin
        // Digit 0
        digit_bitmap[0][0] = 8'b01111110;
        digit_bitmap[0][1] = 8'b01100110;
        digit_bitmap[0][2] = 8'b01100110;
        digit_bitmap[0][3] = 8'b01100110;
        digit_bitmap[0][4] = 8'b01100110;
        digit_bitmap[0][5] = 8'b01100110;
        digit_bitmap[0][6] = 8'b01100110;
        digit_bitmap[0][7] = 8'b01111110;

        // Digit 1
        digit_bitmap[1][0] = 8'b00011000;
        digit_bitmap[1][1] = 8'b00111000;
        digit_bitmap[1][2] = 8'b01111000;
        digit_bitmap[1][3] = 8'b00011000;
        digit_bitmap[1][4] = 8'b00011000;
        digit_bitmap[1][5] = 8'b00011000;
        digit_bitmap[1][6] = 8'b00011000;
        digit_bitmap[1][7] = 8'b01111110;

        // Digits 2-9 would be defined similarly
        // ...
    end

    // Variables for digit display
    wire [3:0] timer_tens, timer_ones;
    wire [3:0] lap_tens, lap_ones;
    wire timer_digit_on, lap_digit_on;
    wire [2:0] timer_digit_row, timer_digit_col;
    wire [2:0] lap_digit_row, lap_digit_col;

    // Extract digits
    assign timer_tens = timer / 10;
    assign timer_ones = timer % 10;
    assign lap_tens = lap_counter / 10;
    assign lap_ones = lap_counter % 10;

```

Main

```
//=====
// Module: vga_sync
// Description: Generates VGA timing signals for 640x480 @ 60Hz
//=====
module vga_sync(
    input clk,          // 100MHz board clock
    input reset,
    output reg hsync,   // Horizontal sync output (active low)
    output reg vsync,   // Vertical sync output (active low)
    output video_on,    // Active video area flag
    output reg [9:0] x, // Pixel column (0 to 799)
    output reg [9:0] y, // Pixel row (0 to 524)
    output p_tick       // Pixel tick pulse at 25MHz (for pixel timing)
);
    //=====
    // Clock Divider: Divide 100MHz -> 25MHz pixel clock
    //=====
    reg [1:0] clk_div;
    always @(posedge clk) begin
        if (reset)
            clk_div <= 0;
        else
            clk_div <= clk_div + 1;
    end
    assign p_tick = (clk_div == 0);

    //=====
    // Horizontal and Vertical Counters
    //=====
    always @(posedge clk) begin
        if (reset) begin
            x <= 0;
            y <= 0;
        end
        else if (p_tick) begin
            if (x == 799) begin
                x <= 0;
                if (y == 524)
                    y <= 0;
                else
                    y <= y + 1;
            end
            else
                x <= x + 1;
        end
    end

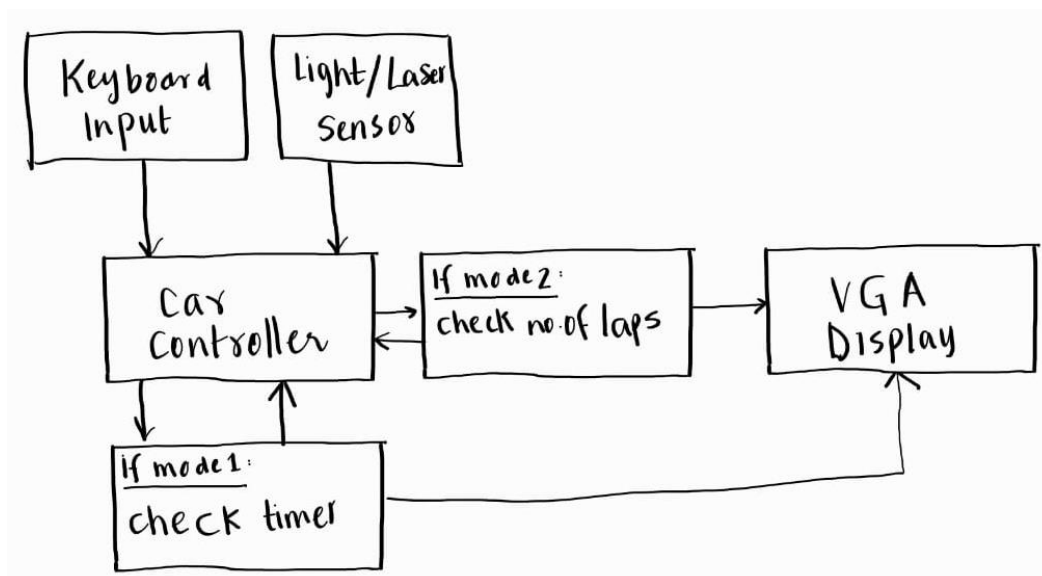
    //=====
    // Generate sync pulses (active low)
    //=====
    always @(posedge clk) begin
        if (reset) begin
            hsync <= 1;
            vsync <= 1;
        end
        else if (p_tick) begin
            // Horizontal sync: active low during pixel columns 656 to 751
            if (x >= 656 && x < 752)
                hsync <= 0;
            else
                hsync <= 1;
            // Vertical sync: active low during rows 490 to 491
            if (y >= 490 && y < 492)
                vsync <= 0;
            else
                vsync <= 1;
        end
    end
end
```

More to be continued

FSM Design

- FSM handles the following states:
 - IDLE: Waiting to start
 - RUNNING: Game in progress
 - FINISHED: Game over, display score
- Transitions depend on timer or laps and checkpoint logic.

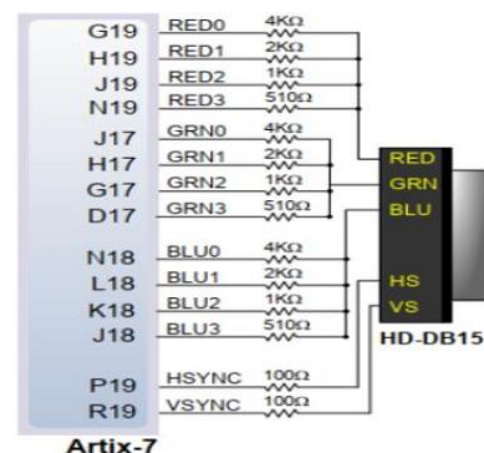
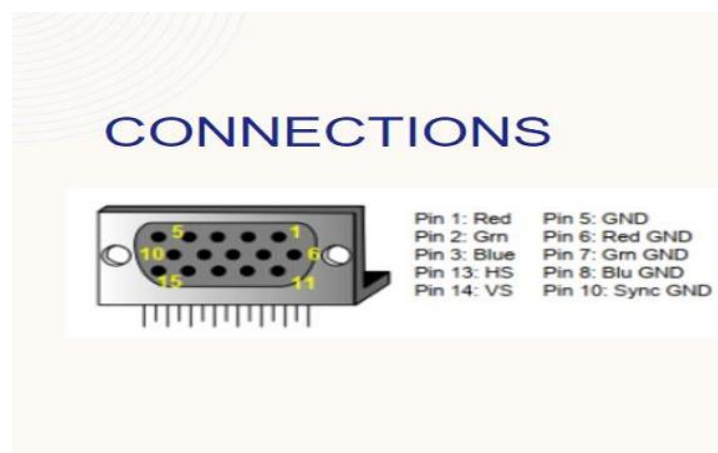
In Overdrive, the game operates with a dynamic start screen that is presented upon loading. The transition from the start screen to the main game is initiated by pressing the "Enter" key. During gameplay, the user can navigate back to the start screen by pressing the "Backspace" key. Furthermore, we can conclude that our game exhibits a **Mealy** Machine behavior by considering user input as a factor in determining the next state. This design choice provides a more interactive and responsive gaming experience.



VGA Output

- Background: Black
- Track: Gray rectangle with borders
- Car: Red and Purple pixel block (8x8)
- Checkpoint: Blue line across track
- Finish Line: Green line at the end

The display screen is generated using the VGA (video graphic array) connector. The display which we are using is standard 640 x 480 pixels. For imaging on the horizontal axis, we turn the video on from 0 till 639 pixels and for the vertical axis the video is on from 0 till 479 pixels as that is the main display. VGAsync was used for turning on the screen and pixel gen are used to create our desired screen. Following pin configuration is used to connect the VGA connector to the FPGA board



Results

The implementation of the car racing game on the Basys 3 FPGA board was successfully completed, with all core functionalities working as intended. The system responds well to user inputs via keyboard, and the graphical output is accurately rendered on a VGA monitor. The car's movement is smooth and responsive, allowing the player to steer through a predefined track displayed on screen.

One of the key results achieved was the real-time movement and tracking of the car, which is controlled using left and right directional keys. The game incorporates a lap counter system that registers each completed lap only when the car passes specific checkpoints in the correct sequence, ensuring fair and consistent lap detection.

A countdown timer of set seconds was also successfully implemented using clock division and FSM-based control logic. Once the time expires, the system freezes gameplay and displays the final score, which reflects the number of laps completed within the time limit. This score is output through seven-segment displays, offering a clear and intuitive readout of player performance.

We also conducted tests on the physical control inputs, including pedal simulations using pushbuttons, and plan to further enhance this with light and laser sensors. These sensors, once fully integrated, will detect

foot motion more precisely, simulating the acceleration and braking experience of an actual car. The keyboard interface continues to serve as the primary input during initial testing, and future iterations will shift toward full sensor-based control for a more arcade-authentic feel.

Furthermore, the FSM (Finite State Machine) design used to control game flow handling states like idle, running, paused, lap complete and game over, demonstrated robust and deterministic transitions, ensuring that the game logic behaved predictably under all test cases.

Overall, the project achieved its goals of combining digital design principles with engaging user interaction. The arcade machine's physical model is under development and will soon include mounted buttons, pedals, sensors, and display elements, turning this into a fully immersive gaming experience. The modularity of the design allows for further enhancements, such as obstacle addition, speed variations, and multiplayer support.

Testing and Simulation

- **Simulation:** Modules were tested in isolation on Vivado using test benches.
- **On Board Testing:** Successfully deployed on Basys 3, verified car movement, lap detection, timer countdown, and display output.
- **Edge Cases Tested:**
 - Reaching finish line before checkpoint → no lap counted
 - Moving outside track → movement blocked
 - Stopping at edge → no movement beyond

Challenges Faced

- Synchronizing car movement and pixel refresh on VGA.
- Avoiding multiple lap counts due to lingering at the finish line.
- Keeping car movement consistent with 25MHz VGA clock.
- Creating an accurate timer with slow display updates.

Future Improvements

- Add obstacles and collision detection.
- Implement multiple difficulty levels (speed increases).
- Add sound effects and score animations.
- Pause functionality using buttons.
- Display time and score on the LCD.
- Build the model structure for display

Conclusion

This project reinforced concepts of FSM, modular design, and timing in Verilog. It provided practical experience in interfacing digital circuits with video output and handling real-time input/output on FPGA hardware. The game works as intended and offers a fun demonstration of what can be built with digital logic design principles.